

Competitive Programming

Section 19: Dijkstra And Weighted Modeling

Competitive Programming Course

2026-07-10

Table of contents

1	Why Dijkstra	3
2	Recognition	4
2.1	Nonnegative is a requirement, not a hint	4
3	Priority queue primer	5
4	Dijkstra template	6
5	The invariant	7
6	Worked example	8
7	Grid Dijkstra	9
8	State expansion	10
9	Complexity	11
10	Implementation notes	12
11	Local exercises	13
11.1	A. Weighted Route Queries	13
11.2	B. Grid Toll Path	13
11.3	C. Discount Route	13
12	Submission checklist	14
13	Exercises	15

1 Why Dijkstra

BFS solves shortest paths when every move costs the same. Once edges have different nonnegative costs, a path with fewer edges may be more expensive:

```
1 -> 2 cost 100
1 -> 3 cost 1
3 -> 2 cost 1
```

BFS would see vertex 2 in one edge. The cheapest route is actually 1 -> 3 -> 2 with cost 2.

Dijkstra's algorithm is the standard tool for shortest paths with nonnegative edge weights.

2 Recognition

Statement signal	Likely tool
shortest path with nonnegative weights	Dijkstra
grid with tolls/costs, not just steps	Dijkstra on cells
one-time option changes cost	Dijkstra on expanded states
all weights are 0 or 1	next section: 0-1 BFS
negative edge weights	not Dijkstra

2.1 Nonnegative is a requirement, not a hint

Before writing a heap, inspect **every transition cost in the graph you are actually going to run on**. Dijkstra requires all of them to be at least zero. That includes costs introduced by a grid model or a state expansion, not merely the numbers visible in the original statement.

For example, suppose the source has edges $s \rightarrow a$ of cost 1 and $s \rightarrow b$ of cost 2, while $b \rightarrow a$ has cost -5. A heap pops and finalizes a at cost 1 before it pops b . But the route $s \rightarrow b \rightarrow a$ costs -3. The finalization claim is false, so ordinary Dijkstra is invalid.

Use Bellman-Ford or a problem-specific method when negative edges are allowed. If negative cycles are relevant, the question may not even have a finite shortest-path answer. Do not try to repair this by changing the visited array: the theorem, not the implementation detail, has failed.

3 Priority queue primer

Dijkstra repeatedly expands the not-yet-final vertex with smallest known distance. A min-priority queue gives that vertex efficiently.

C++ min-heap:

```
priority_queue<pair<long long,int>,  
              vector<pair<long long,int>>,  
              greater<pair<long long,int>>> pq;
```

Python:

```
import heapq  
heapq.heappush(heap, (distance, vertex))  
distance, vertex = heapq.heappop(heap)
```

4 Dijkstra template

```
dist[start] = 0
push (0, start)

while heap is not empty:
    d, u = pop smallest
    if d is not dist[u]:
        continue
    for each edge u -> v with weight w:
        if d + w < dist[v]:
            dist[v] = d + w
            push (dist[v], v)
```

The `if d is not dist[u]` line skips stale heap entries. We do not decrease a key inside the heap; we push a new better pair and ignore the old one later.

5 The invariant

When Dijkstra pops a non-stale pair $(\text{dist}[u], u)$, $\text{dist}[u]$ is final: no future relaxation can make it smaller.

Why nonnegative weights matter (and why the previous counterexample does not contradict it):

Every future path must first leave the heap through a distance $\geq \text{dist}[u]$.
Adding nonnegative edges cannot reduce that distance below $\text{dist}[u]$.

With a negative edge, this reasoning breaks.

6 Worked example

Graph:

```
1 --4--> 2 --7--> 4
1 --2--> 3 --5--> 4
3 --1--> 2
```

Trace:

```
dist[1]=0
pop 1: set dist[2]=4, dist[3]=2
pop 3: improve dist[2]=3, set dist[4]=7
pop 2: no improvement to 4 because 3+7=10
pop stale (4,2): skip
pop 4: final 7
```

7 Grid Dijkstra

A weighted grid is a graph:

```
cell = vertex  
moving into a neighbor adds that neighbor's toll
```

For a cell (r, c) , relax its four neighbors:

```
new distance = dist[r][c] + toll[nr][nc]
```

If the start cell's toll counts, initialize:

```
dist[0][0] = toll[0][0]
```

8 State expansion

Sometimes a vertex alone is not enough state. In Discount Route, the important state is:

```
(vertex, discount_used)
```

There are two layers:

```
layer 0: discount not used yet  
layer 1: discount already used
```

For an edge $u \rightarrow v$ with cost w :

```
(u,0) -> (v,0) cost w  
(u,0) -> (v,1) cost floor(w/2)  
(u,1) -> (v,1) cost w
```

Then run ordinary Dijkstra on this expanded graph.

9 Complexity

For adjacency-list graphs:

```
Time:  $O((n + m) \log n)$   
Memory:  $O(n + m)$ 
```

For two-layer state graphs, the vertex and edge counts are only multiplied by a constant, so the asymptotic complexity stays the same.

10 Implementation notes

Use 64-bit integers for distances in C++:

```
const long long INF = (1LL << 62);
```

In Python, use a large integer sentinel:

```
inf = 10**30
```

Do not mark a vertex visited when it is pushed. Dijkstra may find a better distance before it is popped. The safe moment is when a non-stale pair is popped.

11 Local exercises

11.1 A. Weighted Route Queries

Run Dijkstra once from the source. Answer each target query from `dist`.

11.2 B. Grid Toll Path

Run Dijkstra over grid cells. The cost includes the start cell and every entered cell.

11.3 C. Discount Route

Run Dijkstra over `(vertex, used)` states. The answer is:

```
min(dist[n][0], dist[n][1])
```

12 Submission checklist

```
Are all edge weights nonnegative?  
Did I use a min-heap?  
Did I skip stale heap entries?  
Did I use long long for distances in C++?  
For grids, did I define whether the start cell cost counts?  
For state graphs, did I include every legal transition?
```

13 Exercises

Solve A with duplicate heap entries, B by turning cells into weighted graph states, and C by adding the used-discount bit to the state. Test stale heap entries, an unreachable destination, and a discount used on the first or last edge.